

Data in-memory vs Data on-disk



Files live on disk. We can make the data in them accessible to programs

Dictionary in Python PYnative.com

Unordered collections of unique values stored in (Key-Value) pairs.

```
d = {'a': 10, 'b': 20, 'c': 30}
```

Diagram showing the mapping from keys to values:
↑ ↑ ↑
d['a'] d['b'] d['c']

- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

List in Python PYnative.com

```
L = [ 20, 'Jessa', 35.75, [30, 60, 90] ]
```

Diagram showing the indexing of the list:
↑ ↑ ↑ ↑
L[0] L[1] L[2] L[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Changeable:** List is mutable and we can modify items.
- ✓ **Heterogeneous:** List can contain data of different types
- ✓ **Contains duplicate:** Allows duplicates data

Tuples in Python PYnative.com

```
T = ( 20, 'Jessa', 35.75, [30, 60, 90] )
```

Diagram showing the indexing of the tuple:
↑ ↑ ↑ ↑
T[0] T[1] T[2] T[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Unchangeable:** Tuples are immutable and we can't modify items.
- ✓ **Heterogeneous:** Tuples can contains data of types
- ✓ **Contains duplicate:** Allows duplicates data

Arrays like lists, dictionaries, tuples, live in-memory.
Memory disappears when the program stops running

`id, name, credits`

`1, Alice, 32`

`2, Bob, 28`

```
student_dicts = [  
    {"id": 1, "name": "Alice", "credits": 32},  
    {"id": 2, "name": "Bob", "credits": 28}  
]
```

```
student_lists = [  
    [1, "Alice", 32],  
    [2, "Bob", 28]  
]
```

```
student_tuples = [  
    (1, "Alice", 32),  
    (2, "Bob", 28)  
]
```

Different containers. Same idea: rows and columns.

```
#Create a list of dictionaries that represent the prices of different items  
#along with their serial number in the grocery store.
```

```
myStoreItems = [{345: 'chicken', 567: 'eggs',  
                 561: 'beef', 879: 'mutton'},  
  
                {348: 'milk', 670: 'butter',  
                 127: 'bread', 445: 'tea bags'},  
  
                {237: 'chocolate spread', 381: 'jam',  
                 890: 'sauce', 340: 'sandwich'}]
```

```
print(myStoreItems)
```

```
#accessing the data from the first dictionary  
print("The required item is ",myStoreItems[0][561])
```

```
#accessing the data from the second dictionary  
print("The required item is ",myStoreItems[1][127])
```

```
#accessing the data from the second dictionary  
print("The required item is ",myStoreItems[2][340])
```

```
[{345: 'chicken', 567: 'eggs', 561: 'beef', 879: 'mutton'}, {348: 'milk', 670: 'butter', 127: 'bread', 445: 'tea bags'}, {237: 'chocolate spread', 381: 'jam', 890: 'sauce', 340: 'sandwich'}]  
The required item is  beef  
The required item is  bread  
The required item is  sandwich
```



Built in-memory tables: Fast, Temporary, Flexible, you script the rules

Yahoo Finance Quotes CSV.xlsx - Excel

File Home Insert Page Layout Formulas Data View Design Tell me

I3

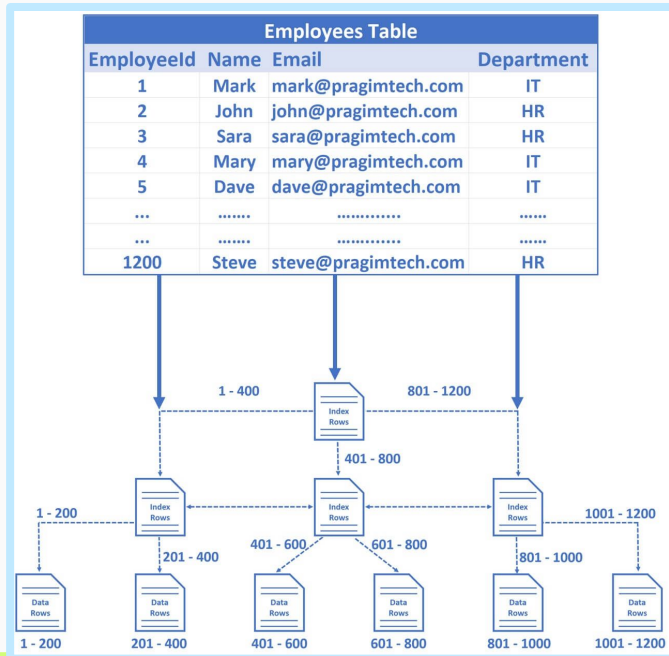
	A	B	D	F	G	H	I	J	K	L	B
1											
2											
3											
4											
5											
6											
7											
8											

Symbol	LastTradeTi	Last	Change	Change	Open	High	Low	Volume
AAPL	2:13:00 PM	142.77	+2.09	1.49%	141.21	142.92	141.16	15,076,485
FB	2:13:00 PM	144.01	+1.74	1.22%	142.95	144.25	142.69	505,956
GOOG	2:13:00 PM	844.65	+6.44	0.77%	841.44	844.78	839.32	551,439
MSFT	2:13:00 PM	65.62	+0.58	0.88%	65.28	65.67	65.14	12,596,063
ORCL	2:13:00 PM	44.51	+0.32	0.72%	44.26	44.55	44.17	5,168,248
YHOO	2:13:00 PM	47.70	+0.70	1.48%	47.25	47.82	47.07	6,056,714

Quotes

Ready

On-Disk Tables (CSV / TSV): Persistent, Shareable, Human-readable, No intelligence



Python SQL Create DB Example

```
import pyodbc
dbConn = pyodbc.connect("Driver={SQL Server Native Client 11.0};"
                        "Server=PRASAD;"
                        "Trusted_Connection=yes;")
```

```
dbCursor = dbConn.cursor()
dbCursor.execute("SELECT name FROM master.dbo.sysdatabases")
```

```
for x in dbCursor:
    print('Database = %r' %x)
```

Python 3.7.1 Shell

tutorialgateway.org

File Edit Shell Debug Options Window Help

```
===== RESTART: C:\Users\sures\Documents\Python\
Database = ('master', )
Database = ('tempdb', )
Database = ('model', )
Database = ('msdb', )
Database = ('ReportServer', )
Database = ('ReportServerTempDB', )
Database = ('SQL Tutorial', )
Database = ('AdventureWorksDW2014', )
Database = ('SQLTEST', )
```

Persistence → survives program end
 Constraints → you manually check ids
 Indexes → dictionary lookup by id